Terminal Buddy: The Master Design Bible

Version: 2.1 (The Expanded Holy Trinity Update) **Date:** February 22, 2026 **Status:** READY FOR CONSTRUCTION / FULLY DETAILED

📖 Table of Contents

1. **The High Concept (The Three Pillars)**
2. **Product Requirements (PRD)**
3. **The Foundation Spec (The Hardware & System Connection)**
4. **The Interaction Spec (Voice Copilot & Controls)**
5. **The Architecture (The System)**
6. **The Roadmap (The Schedule)**

1. 🚀 The High Concept (The Three Pillars)

Standard terminals are ephemeral, context-blind "Walls of Text." When an error occurs and scrolls off the screen, or when a background process silently modifies a file, the developer is forced to play forensic investigator. **Terminal Buddy** is designed to eradicate this cognitive friction. It acts as a true Cognitive Prosthetic for developers, system administrators, and technical creatives.

The system is built entirely upon three unshakeable, deeply integrated pillars:

I. A Beautiful, Multi-Window Terminal (The "Square")

The primary interface is a raw-performance, GPU-accelerated environment that treats text rendering like a high-end video game.

- **Dynamic Tiling & Spatial Memory:** Rather than managing messy overlapping windows, Terminal Buddy supports a dynamic "Square" layout. You can snap multiple terminal sessions into a unified grid. This leverages the human brain's spatial memory—your server logs are always in the bottom right, your build process in the top right, and your active editor on the left.
- **Uncompromising Performance:** Operating at a locked 120FPS with sub-2-millisecond input latency, it eliminates the microscopic lag (the "stutter") found in Electron-based

terminals like VS Code or Hyper. It is visually beautiful, highly legible (with specific considerations for dyslexic-friendly typography), and never drops a frame.

II. The Memvid Sidecar & System Connection

Terminal Buddy does not just blindly log stdout/stderr text; **it connects directly to the computer's nervous system to understand *consequence*.** * **The Flight Recorder:** Every command you type is treated as an "Event." The system tracks the exact moment you press enter and correlates it with actual system effects: Did this command cause a 500MB Disk I/O spike? Did it open a new network port? Did it trigger a silent kernel warning?

- **Deep Time Memory:** This comprehensive "Cause and Effect" telemetry is saved permanently in a `.mv2` (Memvid) file format. This creates a "Deep Time" visual memory of your computer's life. You can literally scroll back six months, find a broken deployment, and see the exact sequence of system-level changes that caused it.

III. The Always-Aware Voice Co-Pilot

Terminal Buddy features an embedded, local AI that constantly watches your active terminal grid and internal system state.

- **Zero-Context-Switching:** You don't have to highlight text, copy it, open a browser, and paste it into ChatGPT. You simply speak or type naturally: *"Buddy, undo that last Git rebase and figure out why the port 8080 is blocked."*
- **Multimodal Execution:** The Co-Pilot inherently understands the context of your screen, reads the background kernel logs we captured, translates your vague intent into precise shell actions, and executes them safely, providing a conversational debrief of what it fixed.

2. 📋 Product Requirements (PRD)

Core Philosophy

- **Go Fast (Zero Friction):** When a developer is in the flow state, the tool must disappear. The UI thread must never block. No garbage collection pauses, no waiting on cloud network latency for basic text rendering. We achieve this exclusively through Rust and GPUI.
- **Remember Everything (Zero Data Loss):** Log every action, every error, and every unseen system change. The user should operate with total psychological safety, knowing that every destructive command is tracked, logged, and ultimately reversible or at least understandable.

- **Analyze Later (Shift-Left Debugging):** Give the user a visual, interactive tool to review their work. When troubleshooting, provide dashboards rather than forcing the user to `grep` through gigabytes of raw text.

Key Features & User Stories

- **P0: The Tiling Rust Terminal:** GPUI + Alacritty Core for instant typing response and multi-pane support.
  - *User Story:* "As a developer, I want to tile 4 instances side-by-side and compile a massive C++ project without my cursor lagging."
- **P0: Deep System Introspection:** Tracking exactly how commands affect the OS in real-time.
  - *User Story:* "As an ops engineer, when a script hangs, I want the terminal to automatically highlight the specific child process and CPU spike causing the hang."
- **P0: The Canvas Dashboard:** A local web UI (`@memvid/canvas-react`) to visually scrub through history and chat with past sessions.
  - *User Story:* "As a team lead, I want to open my dashboard on Friday afternoon and ask 'What dependency changes did I make this week?' and get a clean list."
- **P0: Local Voice Control:** Sub-300ms Whisper.cpp integration for hands-free command execution.
  - *User Story:* "As a user with repetitive strain injury (RSI), I want to hold a hotkey and say 'Run the docker compose file in detached mode' and watch the terminal type and execute it instantly."

3. 🏗️ The Foundation Spec ("The Hardware & System Connection")

This section dictates the "Metal"—the unforgiving technical requirements needed to fulfill the PRD.

The Core Chassis: Rust 🦀

We build entirely in **Rust**. This is a hard requirement. We explicitly reject JavaScript/TypeScript/Electron for the core terminal interface to eliminate Garbage Collection pauses, minimize memory bloat, and ensure raw, predictable speed that respects the user's battery life and CPU cycles.

The Engine Block (Dependencies)

- **Rendering (`gpui`):** Originally developed for the Zed editor, this framework bypasses standard OS window drawing and talks directly to the GPU (Metal on macOS, Vulkan/Wayland on Linux, DirectX on Windows). It guarantees 120FPS rendering of thousands of glyphs.
- **Terminal Logic (`alacritty_terminal`):** The fastest existing terminal core. We ingest Alacritty's VTE (Virtual Terminal Emulator) parsing logic to ensure 100% compatibility with complex terminal applications like `htop`, `tmux`, and `vim`.
- **Process Handling (`portable-pty`):** Ensures flawless, cross-platform pseudo-terminal shell integration.

The System Connection (Hardware Abstraction Layer) 🔌

This is the central nervous system. Terminal Buddy "sees" the effects on the computer by tapping directly into the OS kernel APIs using highly optimized Rust crates.

- **IO/Process Monitoring (`sysinfo` & `procfs`):** Captures memory bloat, CPU spikes, and orphaned zombie processes caused by terminal commands. It snapshots the system state microseconds before and after a command runs.
- **System Logs (`libsystemd` / macOS Unified Logs):** We hook into the OS kernel. If a user runs `npm start` and it fails because of a low-level socket permission issue, Terminal Buddy catches the exact kernel-level error (e.g., `EACCES`) in real-time, even if it wasn't printed to stdout.
- **File Watcher (`notify`):** A high-performance recursive watcher that tracks exactly which files were modified, created, or deleted during a command's execution lifecycle.

The Memory Engine: Memvid 📹

- **Format:** `.mv2` (Video-based Data WAL).
- **Concept:** It acts as a Write-Ahead Log. Every single "frame" appended to this file saves a compressed delta of the Terminal Text grid AND the paired System Connection Data (CPU/Disk/Logs).
- **Portability & Searchability:** The entire memory of your computer's life is stored in a single, append-only file. This file contains embedded vector representations of actions, making it semantically searchable (e.g., searching for the *concept* of "fixing database permissions" rather than exact string matches).

4. 🎮 The Interaction Spec ("Voice Copilot & Controls")

The Voice Engine: "Always-Aware Co-Pilot" 🎙️

Voice integration is not a gimmick; it is treated as a first-class input method equal to the keyboard.

- **Constant Context:** The AI never starts from a blank slate. Because of the Memvid integration, the LLM is constantly fed the last 50 lines of your screen and the active kernel warnings.
- **Privacy-First Pipeline:** Mic -> **Local Whisper.cpp** (Rust bindings) -> Text -> Local/Remote LLM Context Engine -> PTY Injection. Audio is processed 100% locally on-device. No voice data is ever sent to the cloud.
- **Interaction Modes:**
  - **Command Mode (Imperative):** *"List all hidden files sorted by size."* -> The AI translates this to `ls -laS` and executes it immediately.
  - **Fix Mode (Diagnostic):** *"Why did that build just fail?"* -> The AI reads the compiler errors from the terminal grid, correlates them with the Kernel logs, and explains the missing system dependency.
  - **Pair Programming Mode:** *"Watch this script execution and tell me when the memory usage spikes above 2GB."*

The Editor: Neovim Embedded 📝

We actively avoid reinventing the wheel. We do not write a custom text editor. We embed **Neovim** using the `nvim --embed` RPC protocol.

- **How It Works:** The Rust GPUI frontend acts purely as a high-speed "glass" displaying Neovim's internal grid.
- **The Benefit:** You get the industry-standard, high-speed editor inside the terminal. It is instantly compatible with 100% of your existing Neovim plugins, Lua configurations, and keybindings, allowing power users to migrate with zero friction.

5. ⚙️ The Architecture ("The System")

The "Sidecar Pattern" 🏍️

To ensure the terminal remains blistering fast while simultaneously running heavy AI models and disk I/O, we strictly split the application into two isolated processes communicating via a high-speed inter-process communication (IPC) bridge.

Process 1: The GUI (The Driver)

- **Tech:** Rust + GPUI.
- **Responsibility:** Managing pixel rendering, calculating Multi-Window layout boundaries (The "Square"), and capturing raw Keyboard/Mouse/Voice interrupts.
- **The Golden Rule: NEVER BLOCKS.** Under no circumstances is this process allowed to wait on a network request, a heavy disk write, or an LLM inference.

Process 2: The Daemon (The Sidecar)

- **Tech:** Rust + Tokio (Asynchronous Runtime) + Memvid.
- **Responsibility:**
  - Spawning and managing the actual Unix/Windows Shells (`bash`, `zsh`, `nushell`).
  - Continuously polling the System Connection (Kernel Logs, Filesystem watcher).
  - Safely flushing data to the Memvid Logbook on disk.
  - Orchestrating the Voice transcription and LLM API calls.

```
graph TD
    User --> Input[Voice/Keyboard/Mouse]
    Input --> GUI[Rust Multi-Window GPUI]

    %% High-speed Bincode over Unix Domain Sockets
    GUI -- IPC Bus --> Daemon[Sidecar Process]
    Daemon -- Render Updates --> GUI

    subgraph "The Sidecar (Tokio Runtime)"
        Daemon --> Shell[Nushell / ZSH / Bash]
        Daemon --> OS[Kernel Logs & Sysinfo Sensors]
        Daemon --> Memvid[Memvid Writer WAL]
        Daemon --> AI[Whisper & Context Engine]
    end

    Memvid --> Storage[("history.mv2")]

    subgraph "The Local Dashboard"
        Canvas[Canvas React UI] --> API[Canvas Node Server]
        API --> Storage
    end
```

6. 🗺️ The Roadmap ("The Schedule")

We will execute this build in four distinct, verifiable phases. Progression to the next phase strictly requires passing performance and stability criteria from the previous phase.

Phase 1: The Iron Skeleton (Current)

- **Goal:** A running, stable 120FPS Multi-Window GUI that successfully spawns a Shell and accepts text input.
- **Tasks:** * Initialize the Rust Cargo Workspace (`gui` , `daemon` , `shared`).
  - Implement the GPUI application loop and text-grid rendering logic.
  - Integrate `alacritty_terminal` for VTE parsing and `portable-pty` to bind the shell.
- **Acceptance Criteria:** Typing `top` or `htop` inside the terminal renders correctly at 120FPS without visual artifacts or noticeable input lag.

Phase 2: The Control Plane

- **Goal:** Expand input methods to include advanced typing via embedded Neovim and hands-free speaking via Whisper.
- **Tasks:** * Build the `nvim --embed` RPC bridge to map Neovim grid updates to GPUI.
  - Implement `cpal` for low-latency microphone audio buffering.
  - Integrate `whisper.cpp` Rust bindings for on-device transcription.
- **Acceptance Criteria:** User can successfully trigger a hotkey, speak a command, and watch it execute in the shell in under 500ms total round-trip time.

Phase 3: The System Connection & Memory Core

- **Goal:** Connect the terminal to the OS kernel and begin permanently writing the "Black Box" `.mv2` logs.
- **Tasks:** * Implement `sysinfo` pre/post-command snapshot logic.
  - Build platform-specific parsers for system logs (journald/Apple Unified Log).
  - Implement the Memvid writer to serialize grid states and system deltas to disk asynchronously.
- **Acceptance Criteria:** Running a disk-heavy command (e.g., `dd`) automatically appends a frame to `.mv2` accurately reflecting the gigabytes written and the CPU spikes, without stuttering the GUI.

Phase 4: The Visual Brain

- **Goal:** Bring the history to life via the Web Dashboard.
- **Tasks:** * Scaffold a local Next.js application.
  - Integrate `@memvid/canvas-react` components.
  - Build the REST/GraphQL bridge allowing the React app to query the `.mv2` file.
  - Implement the RAG (Retrieval-Augmented Generation) Chat UI for conversing with historical data.